

PROJECT PROPOSAL

FLIGHT IQ

A REAL-TIME BIG DATA ANALYTICS AND
FLIGHT DELAY DETECTION SYSTEM

.....
.....
.....

SUMMER TRAINING PROGRAM: AUGUST - SEPTMPER 2026

TRACK: BIG DATA ANALYSIS

GROUP: S26-B8-BIG DATA-G8-M

Prepared by:

MARWAN KHALED SAYED BORAIY

ABDULLAH HUSSEIN MOHAMMED ELSAYED

ROQAIA GAMAL HOSNY MOHAMED

BASMALA ATEF MOHAMED DARWESH

GitHub Repo Link:

[GITHUB.COM/MARWAN5ALED26/FLIGHT-IQ](https://github.com/MARWAN5ALED26/FLIGHT-IQ)



TABLE OF CONTENTS

1. Executive Summary.

2. Business Problem Statement

3. Project Overview & Concept

4. Tools & Technologies Used

5. Project Workflow & Process

5.1 Storage & Batch Foundation

5.2 Stream Simulation & Ingestion

5.3 Real-Time Processing & ML Inference

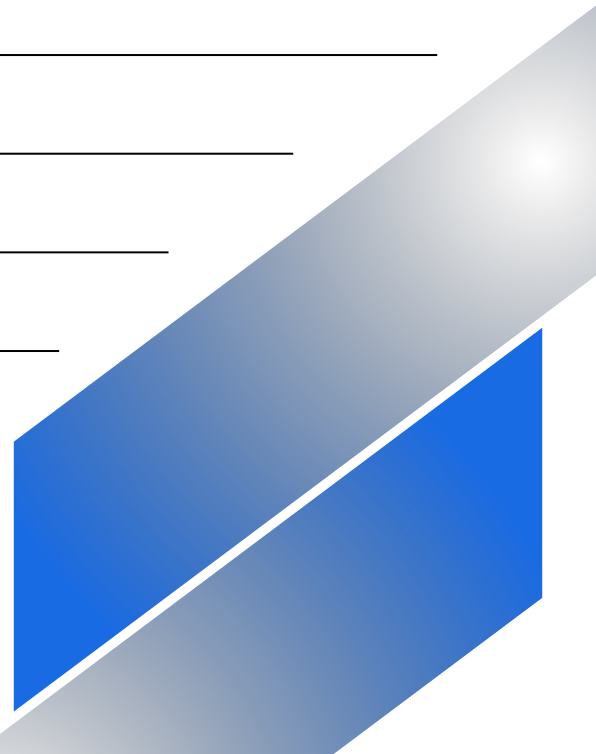
5.4 Visualization

6. System Architecture

8. Functional & Non-Functional Requirements

9. Challenges & Mitigation Strategies

10. Project Deliverables





INTRODUCTION

1. EXECUTIVE SUMMARY

FlightIQ is a Big Data project designed to track and predict flight delays. We use both historical data and live data streams from the Bureau of Transportation Statistics (BTS). By combining tools like Kafka, Spark, HDFS, and PostgreSQL, we aim to turn raw flight data into clear charts and live delay predictions.

2. BUSINESS PROBLEM STATEMENT

Flight delays cause huge problems for airlines and passengers. We are trying to solve two main technical issues:

- Huge Historical Data: Past flight data is simply too big to analyze quickly on a normal computer without Big Data tools.
- Lack of Live Tracking: Most current systems only look at past data and cannot track or predict delays as they happen in real time.

.
.
.

3. PROJECT OVERVIEW & CONCEPT

We are building a system with two main parts: a Batch side for old data and a Streaming side for live data. This system will allow us to:

- Track live flights and catch delays instantly.
- Look at past data to see which airlines or airports have the most delays.
- Use a Machine Learning model to predict if a flight will be late before it even lands

4. TOOLS & TECHNOLOGIES USED

Category	Tool/Technology	Role in Project
Data Storage & Lakehouse	HDFS & PostgreSQL	Storage for historical & streaming flight datasets with SQL querying capabilities.
Message Broker	Apache Kafka	Ingestion of real-time flight event streams with low latency.
Processing & ML Engine	Apache Spark (SQL, Streaming, MLlib)	Execution of batch ETL, real-time stream processing, and ML delay predictions.
Visualization & Reporting	Power BI	Interactive dashboards for monitoring real-time flight metrics and historical analytics.
Infrastructure	Docker	Runs and manages all big data tools (Kafka, Spark, HDFS, PostgreSQL) in isolated environments using a single file.
Version Control	Git & GitHub	Codebase management, collaborative development, and project tracking.

5. PROJECT WORKFLOW

5.1 Storage & Batch Foundation

- Upload Data: Save the raw historical CSV files into HDFS.
 - Clean Data (ETL): Use Spark to clean the data, remove empty values, and save it as Parquet files so it loads faster.
 - Database Setup: Create tables in PostgreSQL so we can easily run SQL queries on the old data.
-

5.2 Stream Simulation & Ingestion

- Simulate Live Data: Write a simple Python script that reads rows from a file and sends them out as if they were live flights happening right now.
 - Kafka: Send this "live" data to Kafka topics to be processed.
-

5.3 Real-Time Processing & ML Inference

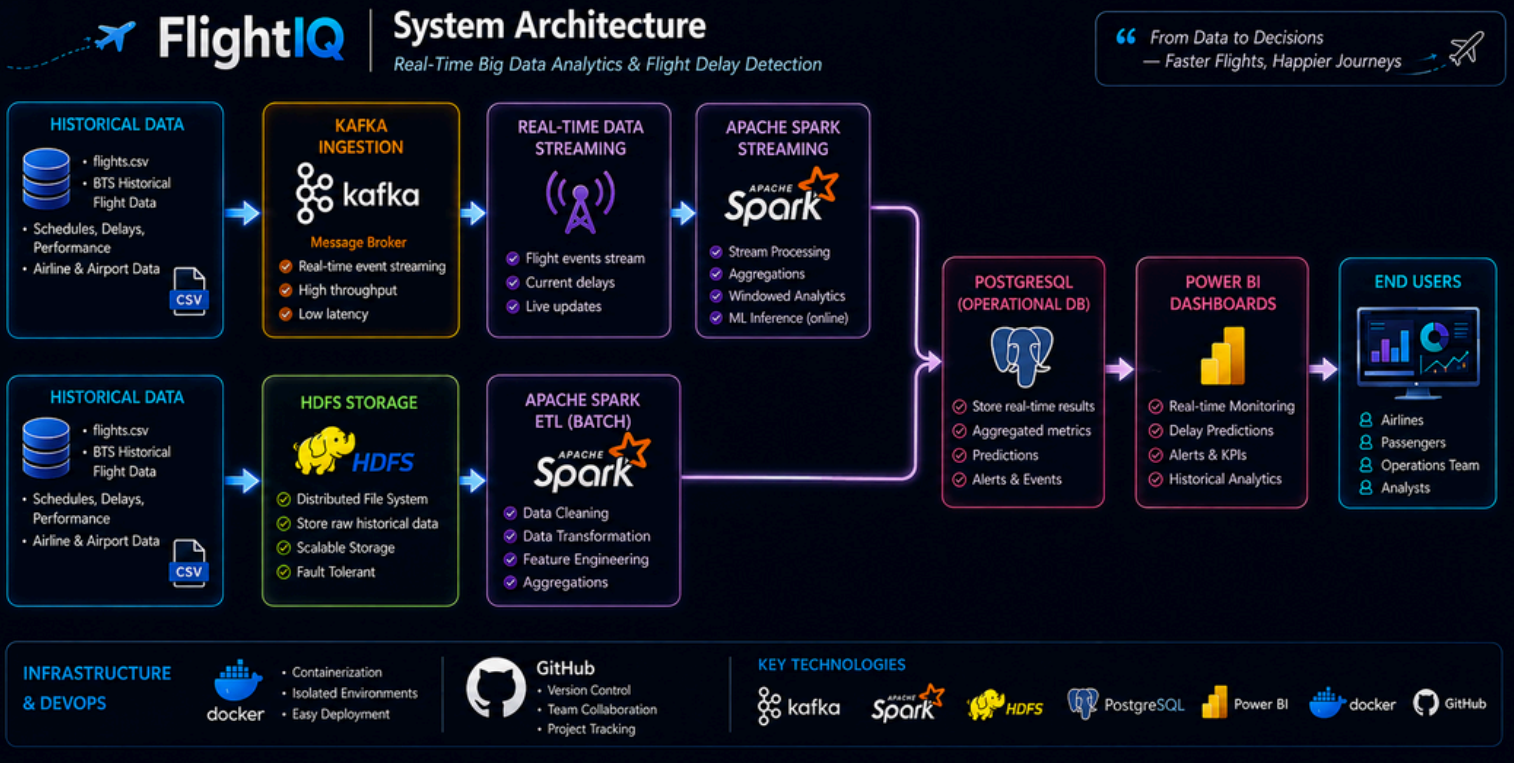
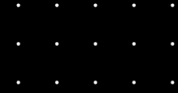
- Process Live Data: Use Spark Streaming to read the data from Kafka and calculate quick stats (like average delays every 5 minutes).
 - Live Predictions: Connect our pre-trained Machine Learning model to this live data to guess which flights will be late.
-

5.4 Visualization

- Dashboard: Show all these stats and predictions on a simple, easy-to-read dashboard from data batch and real-data streaming.
-



6. SYSTEM ARCHITECTURE



ML PIPELINE & REQUIREMENTS

7. MACHINE LEARNING PIPELINE & STRATEGY

- **The Goal:** Predict if a flight will be delayed by more than 15 minutes (Yes/No).
- **Key Features:** We will use data like Time, Distance, Day of the Week, Airline, and Airport.
- **Training the Model:** Train a model (like Random Forest) using Spark MLlib on the old historical data.
- **Using the Model:** Plug this trained model into our live Spark stream to predict delays instantly

8. FUNCTIONAL REQUIREMENTS

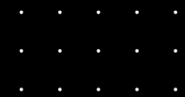
- **Real-Time Stream Processing:** Read JSON flight events from Kafka and process them in Spark using 5-minute windows.
- **Delay Cause Categorization:** Group delays by reason (Carrier, Weather, NAS, Security, LateAircraft) and calculate their percentages.
- **Carrier & Airport Rankings:** Compute On-Time Performance (OTP %) and average delays, saving results to partitioned PostgreSQL tables.
- **Predictive Delay Inference:** Apply a pre-trained ML model to live streams to predict if a flight will be delayed over 15 minutes.

9. NON-FUNCTIONAL REQUIREMENTS

- **Scalability:** Scale horizontally by adding Spark workers, HDFS DataNodes, or Kafka partitions.
- **Performance & Latency:** Maintain end-to-end stream processing under 2 seconds and use Parquet format for fast PostgreSQL queries.
- **Fault Tolerance:** Use Spark checkpointing and HDFS replication to prevent data loss if a node crashes.
- **Portability:** Spin up the entire environment (Kafka, Spark, HDFS, PostgreSQL) using docker-compose up -d on local machines.



CONCLUSION



9. CHALLENGES & MITIGATION STRATEGIES

Challenge	How We Will Fix It
Unbalanced Data: Most flights are on time, so the ML model might get confused.	Use techniques like SMOTE in Spark to balance the data before training.
Missing Values: Canceled or on-time flights have empty delay columns.	Fill empty delay times with "0" and handle canceled flights separately.
Heavy on Computer RAM: Running Kafka, Spark, and Hadoop together is heavy on a local machine.	Fine-tune Docker container limits (e.g., capping Spark Worker memory at 2GB and tuning JVM heap space) and optimize resource allocation across the local cluster.

10. Project Deliverables

- 1. Code:** A GitHub repo with all our Python and Spark scripts.
 - 2. Setup File:** A docker-compose.yml file to run everything with one click.
 - 3. Database:** PostgreSQL tables ready for running queries.
 - 4. Dashboard:** A visual interface showing our results.
 - 5. Report & Video:** A simple document explaining the project and a quick video showing it working.
-